

UNIVERSITÀ DEGLI STUDI DI
MILANO-BICOCCA
ADVANCED MACHINE LEARNING
FINAL PROJECT

Classificazione Fine-Grained su CUB-200-2011

Authors:

Emma Villa - 885906 - e.villa80@campus.unimib.it
Nicolò Molteni - 933190 - n.molteni6@campus.unimib.it

20 febbraio 2026



Sommario

Questo studio affronta la classificazione fine-grained di 200 specie di uccelli. Vengono confrontate tecniche di transfer learning (fine-tuning e feature extraction) su reti pre-addestrate (MobileNetV2, ResNet50V2, InceptionV3) e una CNN custom addestrata da zero. I risultati dimostrano la netta superiorità del fine-tuning sia sull'apprendimento from scratch sia sulla feature extraction con SVM. InceptionV3 si conferma l'architettura ottimale mentre MobileNetV2 offre una valida e leggera alternativa computazionale.

1 Introduzione

La classificazione delle specie di uccelli è un task computazionalmente complesso a causa della bassa varianza inter-classe (specie diverse risultano visivamente simili) e dell'alta varianza intra-classe (differenze di illuminazione, pose, sfondi e occlusioni).

Per risolvere questo problema su un dataset *fine-grained* di 200 specie, il progetto esplora l'uso del transfer learning (sia feature extraction che fine-tuning) impiegando tre reti pre-addestrate su ImageNet. Inoltre è stata sfruttata anche una CNN custom addestrata da zero. I modelli pre-addestrati scelti offrono un interessante confronto strutturale: **MobileNetV2** (3.5M parametri), scelta per la sua leggerezza, funge da baseline per valutare le performance di una rete compatta su un dataset limitato. **ResNet50V2** (25.2M) e **InceptionV3** (23.9M): scelte per testare se modelli significativamente più complessi (circa 7 volte più grandi) possano estrarre pattern migliori in un task *fine-grained*, pur mantenendo un costo computazionale gestibile e compatibile con le risorse a disposizione.

1.1 MobileNetV2

Questa rete è un'evoluzione di MobileNet [1] ed è progettata attorno a blocchi chiamati *inverted residuals with linear bottleneck*. A differenza dei tradizionali blocchi residuali, l'informazione fluisce espandendo un input a bassa dimensionalità verso l'alta dimensionalità, filtrando i dati tramite una convoluzione *depthwise*, e infine ri-proiettando le feature a bassa dimensionalità usando una convoluzione lineare. L'uso di un'attivazione lineare nel bottleneck è cruciale per preservare la capacità rappresentativa della rete. L'ar-

architettura, descritta in [2], utilizza 19 di questi moduli (con un fattore di espansione pari a 6), supportati da layer di Dropout e Batch Normalization.

1.2 ResNet50V2

Appartenente alla famiglia delle reti residuali profonde, si distingue per la sua architettura *pre-activation*, formalizzabile dalle seguenti equazioni:

$$\begin{aligned} \mathbf{y}_l &= h(\mathbf{x}_l) + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l) \\ \mathbf{x}_{l+1} &= f(\mathbf{y}_l) \end{aligned} \tag{1}$$

dove $\mathcal{F}$ è la funzione residuale che incorpora al suo interno le operazioni di Batch Normalization e ReLU [3], $\mathbf{x}_l$ e $\mathbf{x}_{l+1}$ sono rispettivamente l'input e l'output del blocco residuo corrente, $\mathcal{W}$ sono i parametri del modello e y_l è l'output del blocco residuo. In questa versione, sia h sia f fungono da funzioni identità ($f(\mathbf{y}_l) = \mathbf{y}_l$), trasformando l'operazione in una pura addizione: $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l)$. Questo design agevola l'addestramento, poiché è più facile ottimizzare la rete affinché la funzione residuale approssimi lo zero ($\mathcal{F}(\mathbf{x}) = 0$) invece che forzarla a imparare la funzione identità da zero.

Strutturalmente, adotta un design a *bottleneck* in cui ogni blocco residuale è composto da tre convoluzioni consecutive (1×1 , 3×3 , 1×1) per prevenire il problema del *vanishing gradient*.

1.3 InceptionV3

L'architettura InceptionV3 [4] punta a massimizzare profondità e ampiezza minimizzando al contempo il numero di parametri. Il suo principio cardine è la fattorizzazione delle convoluzioni: operazioni spaziali complesse vengono scomposte in sequenze più piccole (ad esempio, un filtro 5×5 viene sostituito da due filtri 3×3 sequenziali).

Un'ulteriore innovazione è la fattorizzazione spaziale asimmetrica, dove una convoluzione $n \times n$ viene sostituita da una sequenza $1 \times n$ seguita da una $n \times 1$, riducendo il costo computazionale senza sacrificare la capacità espressiva della rete.

Infine, la rete ottimizza la riduzione della griglia spaziale parallelizzando stride e pooling (evitando così colli di bottiglia rappresentazionali) e mitiga il problema del *vanishing gradient* impiegando classificatori ausiliari dotati di Batch Normalization.

1.4 Valutazione dei risultati

Le performance di ciascun modello sono misurate attraverso le seguenti metriche, calcolate usando la strategia *macro* dato che il dataset è sbilanciato.

Top-1 Accuracy è la metrica standard per la classificazione. Misura la percentuale di campioni per i quali la classe predetta con la probabilità più alta coincide esattamente con l'etichetta reale (*ground truth*).

$$\text{Top-1 Accuracy} = \left(\frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = \hat{y}_i) \right) \cdot 100 \quad (2)$$

dove N è il numero dei campioni, y_i la classe reale e $\hat{y}_i$ la classe predetta e $\mathbb{I}$ è la funzione di verità.

Top-5 Accuracy considera una predizione corretta se la classe reale è inclusa nell'insieme delle cinque classi a cui il modello ha assegnato le probabilità maggiori. In problemi di classificazione complessi con un elevato numero di classi, il modello potrebbe confondersi tra categorie semanticamente simili.

Recall misura la capacità del classificatore di individuare correttamente tutti i campioni appartenenti a una specifica classe. È definita come:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

dove TP sono i True Positive e FN i False Negative. In un contesto di classificazione *fine-grained* è cruciale che il modello sia in grado di discriminare istanze simili appartenenti alla stessa classe.

Precision è definita come:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

dove FP sono i False Positive. Offre una valutazione della capacità del modello nell'identificazione corretta della classe target all'interno di un ampio insieme di categorie simili.

F1-Score è la media armonica tra Precision e Recall, è essenziale per valutare la robustezza del modello nel discriminare tra categorie visivamente simili.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (5)$$

L’F1 Score garantisce che il modello mantenga un equilibrio ottimale nel distinguere i dettagli fini senza perdere capacità di generalizzazione.

ROC curve (Receiver Operating Characteristic) mostra la capacità del classificatore nel discriminare le classi al variare della soglia di decisione. Mette in relazione il True Positive Rate (Recall) con il False Positive Rate:

$$FPR = \frac{FP}{FP + TN} \quad (6)$$

L’Area sottesa dalla Curva (AUC) fornisce una misura scalare della separabilità delle classi.

Precision-Recall curve (PR Curve) rappresenta l’andamento della Precision rispetto alla Recall. In una valutazione *One-vs-Rest*, dove i campioni negativi superano di gran lunga quelli positivi, la curva PR evita che le prestazioni vengano sovrastimate.

2 Dataset

Per il progetto è stato utilizzato il dataset Caltech-UCSD Birds-200-2011 [5], riconosciuto come benchmark di riferimento per task fine-grained, che mirano a distinguere sottocategorie molto simili tra loro.

Il dataset è un’estensione della versione CUB-200 e si compone di 11.788 immagini appartenenti a 200 classi di specie di uccelli, prevalentemente nord-americani. Come documentato dagli autori, il processo di costruzione del dataset è composto da due fasi: la prima è la raccolta di immagini tramite la piattaforma Flickr, la seconda è il filtraggio con la conseguente annotazione mediante crowd-sourcing su Amazon Mechanical Turk, in modo da garantire un’elevata qualità delle etichette. Il dataset CUB-200-2011 è caratterizzato da due difficoltà principali:

1. Bassa varianza inter-classe: nel dataset sono presenti diverse tipologie di specie di uccelli (es: 21 tipologie di passero) le cui differenze morfologiche sono spesso minime. Come visibile in Figura 1.
2. Alta varianza intra-classe: gli esemplari della stessa specie possono apparire in modo molto diverso in quanto, come evidenziato nel paper, il dataset contiene immagini in cui gli uccelli sono in varie pose diverse,



Figura 1: Esempio della varianza inter-classe dei passeri

ad esempio: in volo o appollaiati. Inoltre in alcune immagini gli uccelli sono parzialmente occlusi dalla vegetazione.

2.1 Suddivisione dei dati

La suddivisione del dataset impiegata è lo split fornito dagli autori, nonostante la proporzione del 50% tra le istanze di training e test. Le metriche durante l'addestramento sono state monitorate grazie al validation set, che è stato creato estraendo il 15% dei campioni appartenenti al Training Set originale. La partizione finale dei dati è la seguente: *Test Set* 5.794 immagini, *Training Set* 5.094 immagini, *Validation Set* 900 immagini.

Questo splitting dei dati aumenta ulteriormente la difficoltà del task a causa della scarsità di dati nel training set. Infatti, dopo aver creato il validation set il modello ha in media solo 25 immagini per classe per svolgere l'addestramento. Tipicamente le Reti Neurali Convoluzionali (CNN) profonde necessitano di un numero maggiore di immagini per eseguire un buon addestramento. La mancanza di dati implica la necessità di utilizzare tecniche di Transfer Learning e Data Augmentation per prevenire l'overfitting.

L'analisi quantitativa delle classi in Figura 2 ha evidenziato un lieve sbilanciamento del dataset. Il numero di immagini per classe varia tra un minimo di 41 e un massimo di 60, di conseguenza è stato ritenuto gestibile senza l'applicazione di tecniche di oversampling.

2.2 Elaborazione e Arricchimento

Al fine di rendere i dati grezzi compatibili con i requisiti delle architetture neurali scelte è stata applicata una pipeline specifica composta da tre fasi: pre-elaborazione, data augmentation e normalizzazione.

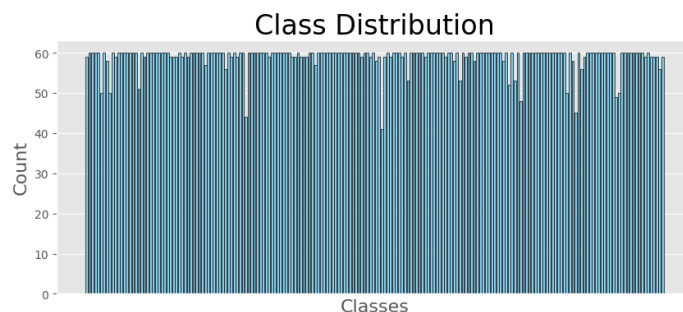


Figura 2: Distribuzione delle classi all'interno del dataset.

Pre-elaborazione: dato un campione di 1000 immagini preso dal training set è stata eseguita un'analisi preliminare sulle loro dimensioni. La Figura 3 mostra una grande varianza nelle dimensioni delle immagini. Queste ultime sono state standardizzate per rendere le immagini compatibili con gli input delle reti neurali. I passaggi applicati sono i seguenti: **Rescaling**, l'immagine viene ridimensionata, rendendo il lato più corto pari a 256 pixel. **Center Crop**, viene ritagliata la parte centrale con una dimensione 224 x 224.

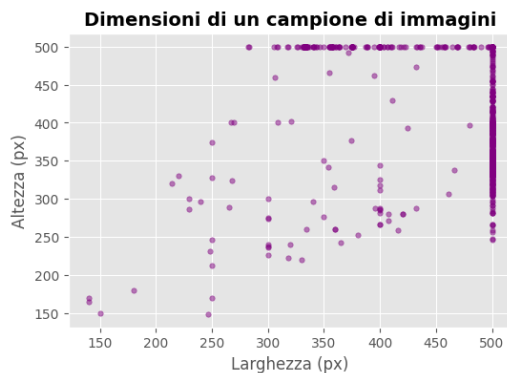


Figura 3: Grafico dimensioni di un campione di immagini

Il processo descritto permette di mantenere inalterata la morfologia dell'uccello, ma comporta una perdita di informazioni sui bordi dell'immagine, che di solito non contengono parti fondamentali per riconoscere un uccello.

Data Augmentation: considerato il problema introdotto dal numero limitato di immagini nel training set è stata progettata una pipeline di Data Augmentation probabilistica integrata nell'architettura del modello.

Invece di adottare un approccio deterministico, è stato introdotto il layer personalizzato *RandomApply*, che applica un set di trasformazioni '*strong*' con una probabilità del 70% ($p = 0.7$). Questa strategia di regolarizzazione bilancia l'apprendimento: l'esposizione a immagini fortemente aumentate favorisce l'estrazione di feature robuste e invarianti, mentre utilizzare immagini originali preserva i dettagli fini necessari al riconoscimento del piumaggio.

La pipeline è composta da: un Random Flip Orizzontale, eseguito sempre, e dalla pipeline di Augmentation '*strong*' contenente i seguenti blocchi:

- **Rotazione** (fattore $0,1 \approx \pm 36^\circ$) e **Shear** (fattore 0,4), simulano variazioni di prospettiva e di posa dell'uccello.
- **Random Zoom**, da -30% a $+20\%$, rende la rete invariante alla distanza dell'uccello, simulando sia un allontanamento che un avvicinamento.
- **Traslazione** sia in orizzontale che in verticale del 15%: Per evitare che la rete si adatti a dove si trova l'uccello nell'immagine.
- **Random Contrast** (fattore 0.2) e **Random Brightness** (fattore 0.2), variano nell'immagine il contrasto e la luminosità per simulare le diverse condizioni di illuminazione.

Tutte le trasformazioni geometriche sono state utilizzate con modalità di riempimento `fill_mode="reflect"`. In modo da mantenere una continuità visiva e non usare un riempimento nero.

Normalizzazione e One-Hot encoding: dopo aver applicato la pipeline di augmentation, è stata eseguita una normalizzazione delle immagini mappando i dati dall'intervallo $[0, 255]$ al range $[-1, 1]$ tramite la trasformazione lineare in Equazione 7. La normalizzazione è necessaria perché i modelli neurali utilizzati (MobileNetV2, ResNet50V2 e InceptionV3) sono stati pre-addestrati su ImageNet e si aspettano di ricevere in input immagini normalizzate nell'intervallo $[-1, 1]$. Infine è stata realizzata una One-Hot encoding delle label, quest'operazione è necessaria sia per compatibilità con la loss function impiegata che per la funzione di output della rete.

$$x_{norm} = \frac{x}{127.5} - 1 \quad (7)$$

3 Approccio Metodologico

Per garantire la riproducibilità degli esperimenti è stato fissato il random seed a 42 per tutte le librerie utilizzate (Numpy, Random, Keras).

3.1 Fine Tuning

Per i modelli MobileNetV2, ResNet50V2 e InceptionV3, viene utilizzata una strategia di addestramento a due fasi, in modo da massimizzare l'efficacia del Transfer Learning e preservare la conoscenza pregressa acquisita su ImageNet, evitando che l'aggiornamento dei pesi distrugga le feature precedentemente apprese.

Head-Training nella fase iniziale, il corpo della rete (backbone) viene mantenuto congelato, quindi i suoi parametri non sono addestrabili. Il training viene eseguito solo sulla testa di classificazione per 5 epoche con un alto learning rate. Questa fase è cruciale, in quanto una backpropagation sull'intera rete genererebbe un gradiente con valore elevato che distruggerebbe i filtri convoluzionali pre-addestrati.

Fine-Tuning in questa fase si procede a sbloccare parzialmente la backbone. Per tutti e tre i modelli viene applicata una logica di sblocco solo dei layer più profondi, seguendo il principio che i primi layer catturino feature generiche, mentre gli ultimi codifichino forme complesse. I layer di Batch-Normalization non sono stati riaddestrati, in questo modo si mantengono le capacità raggiunte durante il training su ImageNet.

Regolarizzazione dei modelli sono state utilizzate un insieme di tecniche di regolarizzazione per mitigare il problema dell'overfitting e massimizzare la generalizzazione. A MobileNetV2 e InceptionV3 è stato applicato un Weight Decay (L_2) di 10^{-3} e un learning rate di 10^{-3} . Mentre per ResNet50V2, a causa della maggior profondità della rete e quindi a una maggiore probabilità di overfitting, è stata applicata una regolarizzazione più aggressiva ($5 * 10^{-3}$) combinata con un learning rate iniziale di $5 * 10^{-4}$.

È stato implementato uno scheduler Cosine Decay, in modo da modulare il learning rate secondo una funzione coseno, che decresce in modo fluido fino ad un limite asintotico minimo. Infine è stato utilizzato l'Early stopping, con patience 10, in modo da fermare l'addestramento quando la loss di validazione smette di migliorare.

3.2 Feature Extraction con SVM

Dopo aver eseguito il fine-tuning delle architetture, si è scelto di confrontare le prestazioni ottenute con un altro approccio classico del transfer learning:

la feature extraction. L'architettura scelta per estrarre le feature è stata InceptionV3 per diversi motivi. In primo luogo, genera un feature vector della stessa dimensione di ResNet50V2 (2048), che risulta 1.6 volte più ampio rispetto al feature vector di MobileNetV2 (1280). In secondo luogo, come verrà mostrato successivamente, ha ottenuto prestazioni migliori rispetto alle altre reti neurali nella fase di fine-tuning. Infine, InceptionV3 impiega filtri convoluzionali di diverse dimensioni (1×1 , 1×7 , ...) applicati in parallelo, di conseguenza, la rete riesce a catturare feature su scale multiple in modo più semplice ed efficace.

L'estrazione delle feature è stata eseguita con due tecniche differenti: nella prima è stato impiegato InceptionV3 pre-addestrato su ImageNet, nella seconda è stato sfruttato InceptionV3 dopo aver eseguito il fine-tuning. In questo modo è possibile osservare se la rete riesce ad estrarre feature ottimali per classificare gli uccelli, inoltre permette di confrontare le prestazioni del classificatore neurale con il classificatore SVM, lineare, a cui è stata applicata anche una `GridSearch` per la ricerca degli iperparametri migliori.

3.3 Architettura Custom CNN

La rete convoluzionale implementata è composta da 4 layer convoluzionali che espandono progressivamente il numero di filtri, ogni convoluzione è seguita da un layer di MaxPooling per ridurre la dimensionalità spaziale delle feature. Infine, sono presenti due layer densi: il primo è composto da 256 neuroni con funzione di attivazione ReLU, il secondo sfruttato per la classificazione contiene 200 neuroni e impiega la funzione Softmax.

Sia la struttura della rete sia gli iperparametri di addestramento sono stati determinati automaticamente tramite Keras Tuner, che permette di esplorare lo spazio di ricerca per identificare la combinazione architetturale in grado di massimizzare la performance sul validation set. Tuttavia, la configurazione suggerita dall'algoritmo è stata lievemente modificata per migliorare il bilanciamento tra complessità computazionale e capacità di generalizzazione. Al fine di contrastare il problema dell'overfitting, sono state integrate tecniche di regolarizzazione e normalizzazione: *Batch Normalization*, *Dropout* e la *regolarizzazione L_2* , utile a penalizzare la magnitudine dei pesi più grandi. Inoltre, è stato applicato il *label smoothing* per evitare che il modello diventi eccessivamente confidente, cercando probabilità troppo nette (*hard probabilities*) durante la fase di training.

4 Risultati

La sintesi dei risultati è riportata nella Tabella 1, e confronta le architetture in termine di accuratezza, tempi di addestramento e funzione di costo dove disponibile. Nonostante la classificazione fine-grained sia un task complesso, soprattutto con il dataset split impiegato, i risultati ottenuti sono stati abbastanza soddisfacenti.

Tabella 1: Confronto tra: loss, top-1 accuracy e tempo di addestramento ottenuti dai modelli testati. I tempi sono riportati usando notazione decimale e non sessagesimale

Modello	Accuracy		Loss		Tempo
	Training	Test	Training	Test	
MobileNetV2	0.659	0.693	1.249	1.122	14.65 min
ResNet50V2	0.757	0.694	0.987	1.132	23.08 min
InceptionV3	0.811	0.758	0.817	0.877	16.60 min
Feature extraction	-	0.662	-	-	8.47 min
Feature extraction fine-tuned	-	0.766	-	-	4.77 min
Custom CNN	0.303	0.312	3.588	3.632	42.62 min

La Tabella 1 mostra che le tecniche di *Transfer Learning* superano nettamente l'approccio from *scratch*. La Custom CNN presenta limiti nell'addestramento su un dataset così piccolo e complesso raggiungendo una accuracy del 31.2%. InceptionV3 ha ottenuto i risultati migliori in assoluto, confermando l'efficacia della sua architettura nel catturare i dettagli fini. MobileNetV2 e ResNet50V2, nonostante la notevole differenza in termini di parametri, hanno raggiunto livelli di accuratezza comparabili. L'approccio di Feature Extraction su InceptionV3 ha fornito una buona baseline, raggiungendo il 66.2%, che sale al 76.6% applicandolo al modello già sottoposto a fine-tuning. Il tempo riportato per quest'ultimo approccio si riferisce alla fase di estrazione, il tempo complessivo risulta circa di 21 minuti.

Il grafico in Figura 4 mostra una comparazione tra tutte le metriche puntuali sfruttate. Inoltre, permette di valutare immediatamente se un modello è sbilanciato verso una specifica metrica o se le prestazioni sono uniformi. Il grafico mostra che: InceptionV3 e InceptionV3-fine-tuned-SVM sono superiori a tutti gli altri modelli testati, questo è osservabile dagli ampi poligoni ottenuti. Inoltre, tutti i modelli presentano un buon equilibrio tra Precision e Recall, questo si può notare dalla regolarità dei poligoni. Tuttavia va esclusa da quest'ultima osservazione la Top-5 Accuracy che è per definizione più sbilanciata.

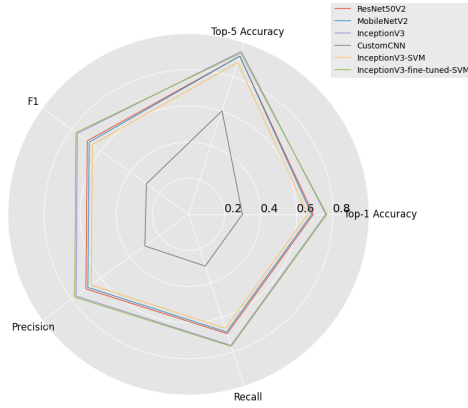


Figura 4: Grafico Radar per il confronto delle metriche puntuali (Accuracy, Top-5 Accuracy, Recall, Precision, F1 score) ottenute da ciascun modello.

La curva ROC della Figura 5 mostra che i modelli di Transfer Learning massimizzano l'area sottesa dalla curva (AUC), che risulta essere vicina all'1. Di conseguenza i modelli assegnano la maggior parte delle volte un punteggio di confidenza più alto alla classe corretta.

La curva PR di Figura 5 mostra che InceptionV3 e InceptionV3-fine-tuned-SVM riescano ad avere una classificazione che non generi molti falsi positivi. MobileNetV2 e ResNetV2 mostrano un decadimento più veloce all'aumentare della Recall, di conseguenza il modello trova maggiori difficoltà nella classificazione di tutti i True Positive.

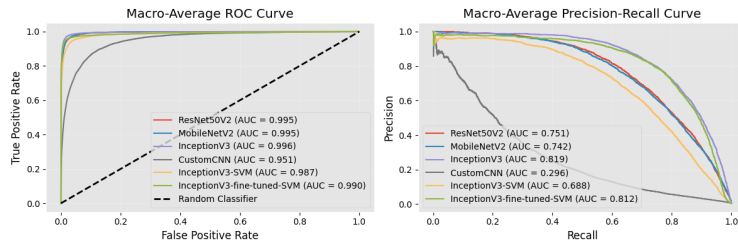


Figura 5: Curva ROC e curva PR dei modelli analizzati.

5 Discussione

La classificazione *fine-grained* è un task intrinsecamente complesso, a maggior ragione considerando lo split del dataset impiegato in questo studio.

Tuttavia, la metodologia di analisi adottata potrebbe essere ulteriormente affinata seguendo quattro direzioni di sviluppo principali. La prima riguarda l'introduzione dei pesi per le classi (*class weights*) durante la fase di training, questo permetterebbe di gestire esplicitamente lo sbilanciamento del dataset. In secondo luogo, il *crop* delle immagini potrebbe essere ottimizzato: all'interno del progetto è stato eseguito un ritaglio centrale (*center crop*), assumendo che l'uccello fosse sempre posizionato al centro della foto. Eseguire un ritaglio mirato, sfruttando le *bounding box* già fornite all'interno del dataset, porterebbe a un miglioramento delle prestazioni. La terza area di intervento consisterebbe nell'adottare una *data augmentation* più spinta e complessa: l'integrazione di tecniche avanzate come il *CutMix* potrebbe infatti migliorare notevolmente la capacità di generalizzazione della rete. Infine, adottare una partizione (*split*) differente del dataset aumenterebbe sicuramente le prestazioni complessive dei modelli, riducendo l'overfitting.

6 Conclusioni

Il progetto svolto ha analizzato l'efficacia di diverse architetture di Deep Learning per la classificazione fine-grained di specie di uccelli. L'analisi ha dimostrato la netta superiorità del Transfer Learning rispetto all'addestramento partendo da zero, infatti la Custom CNN, in 150 epoche è arrivata solo al 32,8% evidenziando i limiti dell'apprendimento senza conoscenza pregressa. In particolare, il confronto tra le metodologie di Transfer Learning ha evidenziato che la semplice Feature Extraction (SVM), pur fornendo una buona base con Test Accuracy=66.2%, rimane inferiore al Fine-Tuning. Di conseguenza viene confermato che per distinguere sottospecie molto simili è necessario ri-addestrare pesi profondi per fare in modo che la rete si specializzi su dettagli morfologici precisi.

Tra le architetture InceptionV3 si è confermata la soluzione ottimale con Test Accuracy pari a 75.8%. Questo risultato è stato ottenuto grazie alla capacità dei moduli di *Inception* nell'elaborare dettagli e forme globali simultaneamente. Tuttavia, MobileNetV2 ha offerto prestazioni competitive con un costo computazionale inferiore, mostrandosi una valida alternativa in caso di risorse limitate. L'integrazione delle bounding box per focalizzare l'attenzione sul soggetto e l'adozione di tecniche di augmentation avanzate come CutMix rappresentano i possibili sviluppi futuri per incrementare le performance dei modelli.

Riferimenti bibliografici

- [1] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “Mobilenets: Efficient convolutional neural networks for mobile vision applications,” 2017. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “Mobilenetv2: Inverted residuals and linear bottlenecks,” 2019. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Identity mappings in deep residual networks,” 2016. [Online]. Available: <https://arxiv.org/abs/1603.05027>
- [4] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, “Rethinking the inception architecture for computer vision,” 2015. [Online]. Available: <https://arxiv.org/abs/1512.00567>
- [5] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The Caltech-UCSD Birds-200-2011 Dataset,” 2011. [Online]. Available: <https://authors.library.caltech.edu/records/cvm3y-5hh21>